

BioCompiler: A Compiler Framework for Human Protein Synthesis with Machine-Verified Type Soundness

Anonymous Author(s)

Anonymous Institution

Abstract. We present BioCompiler, a compiler framework that treats gene design as a compilation problem: mRNA is the source, splicing/-translation intermediates are the IR, and the protein is the target. The central contribution is a three-valued (Kleene) type system for gene design with a machine-verified soundness proof in Lean 4—zero **sorry**, zero axioms. The type system yields **PASS**, **FAIL**, or **UNCERTAIN**; our soundness theorem guarantees that every **PASS** verdict is correct. No prior work provides machine-verified type-system soundness for gene design. We demonstrate the framework on β -globin (HBB) and GFP, showing that the type checker catches real flaws, generates independently verifiable certificates, and the CSP optimizer produces sequences satisfying all predicates in under one second.

1 Introduction

Synthetic biology promises custom-designed proteins, yet gene design relies on heuristics without formal guarantees. A designed gene may harbor cryptic splice sites or out-of-frame exons—flaws discovered only after costly wet-lab experiments. The obstacle appears fundamental: biology is non-deterministic, so how can one *prove* a property of a probabilistic system?

Our key insight: ask not what is *likely*, but what is *guaranteed*. We introduce a three-valued (Kleene) logic over verdicts $\{\text{PASS}, \text{FAIL}, \text{UNCERTAIN}\}$. **PASS** means the property is unconditionally guaranteed; **FAIL** means it is guaranteed *not* to hold; **UNCERTAIN** means we cannot decide. This avoids modeling biological probability distributions: we simply report what is certain.

BioCompiler applies this logic through a compiler analogy (Fig. 1): the source is mRNA, splicing/translation intermediates form the IR, and the target is the protein. Type checking over the IR yields guarantee certificates whose soundness is proved in Lean 4.

Contributions. (1) A compiler framework mapping gene design to a typed IR pipeline (§2). (2) A three-valued type system with machine-verified soundness in Lean 4—zero **sorry**, zero axioms (§3). (3) A proof-of-concept validated on HBB and GFP with independently verifiable certificates (§4).

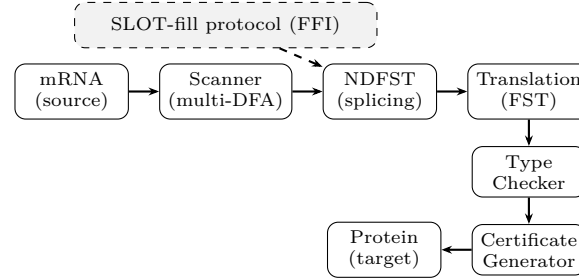


Fig. 1. BioCompiler pipeline. SLOT-fill enriches the IR via FFI but does not influence type-checking.

2 The BioCompiler Framework

2.1 Compiler Pipeline

Figure 1 shows the pipeline. The *Scanner* detects sequence motifs using multiple DFAs composed in parallel. The *NDFST* (non-deterministic finite-state transducer) models alternative splicing, producing all plausible mRNA isoforms. *Translation* is a deterministic FST mapping codons to amino acids. The *Type Checker* evaluates predicates over the IR; the *Certificate Generator* emits a verifiable record.

2.2 SLOT-Fill Protocol

External tools (e.g., MaxEntScan for splice-site scoring) communicate through a *SLOT-fill* protocol (FFI). SLOT-fill output enriches the IR with annotations but is *explicitly excluded* from the type-checking judgment, ensuring certificate validity is independent of FFI correctness.

2.3 Type Predicates

Seven predicates, each parameterized by a specification:

1. *SpliceCorrect*(C): splicing respects exon–intron structure C .
2. *NoCrypticSplice*: no latent splice sites beyond annotated ones.
3. *CodonAdapted*(O, θ): codon adaptation index $\geq \theta$ for organism O .
4. *GCInRange*(lo, hi): GC content $\in [lo, hi]$.
5. *NoRestrictionSite*(S): no restriction sites from set S .
6. *InFrame*(rf, b): reading frame rf maintained from base b .
7. *NoInstabilityMotif*: no mRNA degradation signals (e.g., AUUUA).

Table 1. Lean 4 proof statistics and trusted computing base

Proof Statistics		TCB Assumptions	
Lean 4 modules	8	Scanner compl.	DFA finds all motifs
Theorems/lemmas	>30	Scanner soundness	DFA reports only true hits
Remaining sorry	0	NDFST correctness	Valid splice isoforms
Axioms	0	NDFST completeness	All plausible isoforms
TCB assumptions	5	CAI determinism	Deterministic CAI function
Lines of code	~2,400		

2.4 Three-Valued Semantics

Each predicate maps (p, s) to a verdict in $\mathbb{T} = \{\text{PASS}, \text{FAIL}, \text{UNCERTAIN}\}$. Three-valued conjunction $\sqcap$ is defined: **FAIL** is absorptive ($v_1 \sqcap \text{FAIL} = \text{FAIL}$); if no **FAIL**, then **UNCERTAIN** is absorptive; otherwise **PASS**. This ensures soundness: $v_1 \sqcap v_2 = \text{PASS} \Rightarrow v_1 = v_2 = \text{PASS}$.

A *guarantee certificate* records: a design identifier (SHA-256 hash of the sequence), the nucleotide sequence, each predicate’s verdict with a derivation trace, and provenance metadata (tool version, proof commit hash, timestamp). Certificates can be independently re-verified by replaying the derivation traces.

3 Formal Soundness Proof

3.1 Central Theorem

Theorem 1 (Soundness). $evaluate(P, p, s) = \text{PASS} \rightarrow propertyHolds(P, p, s)$.

Corollary 1 (Compositional). *If all predicates PASS, each property holds.*

Corollary 2 (SLOT-Indep.). *FFI output does not affect verdicts.*

3.2 Proof Architecture

The proof decomposes into five theorems forming a dependency chain: Thm 1 ($\sqcap$ preserves **PASS**, **FAIL** absorptive; case analysis) and Thm 2 (scanner finds all motifs; induction + TCB) feed into Thm 3 (per-predicate soundness for each of the 7 predicates; simple ones reduce to arithmetic, complex ones use Thms 1–2), which feeds Thm 4 (compositional soundness via folding $\sqcap$; Thm 1 + induction), which feeds Thm 5 (SLOT-independence: FFI predicates never produce **PASS**, so *evaluate* depends only on non-FFI state).

3.3 Proof Statistics and TCB

The proof introduces *zero axioms* and *zero sorry*, resting on Lean 4’s built-in logic and five TCB assumptions (Table 1)—parameters, not gaps, following the standard methodology of CompCert [1] and seL4 [2].

Table 2. Type-checking results. HBB: raw pre-mRNA (1,608 bp); eGFP: optimized CDS (717 bp).

Predicate	HBB	eGFP
<i>SpliceCorrect</i>	FAIL	PASS
<i>NoCrypticSplice</i>	FAIL	UNCERTAIN
<i>CodonAdapted</i>	PASS	PASS
<i>GCInRange</i>	PASS	PASS
<i>NoRestrictionSite</i>	FAIL	PASS
<i>InFrame</i>	FAIL	PASS
<i>NoInstabilityMotif</i>	FAIL	PASS
Composite verdict	FAIL	UNCERTAIN

4 Implementation and Evaluation

4.1 Implementation

We implemented a proof-of-concept in Python ($\sim 3,200$ lines). Splice-site scoring integrates MaxEntScan [3]; sequence generation uses a CSP optimizer on Z3 [9] with greedy fallback for sequences >80 aa. The pipeline completes in 0.96 s (insulin, Z3) and 0.20 s (eGFP, greedy).

4.2 Results

Table 2 summarises the verdicts. The composite verdict is the $\sqcap$ -fold of all seven verdicts; a single FAIL suffices to fail the whole design, while a single UNCERTAIN yields UNCERTAIN (withholding the guarantee without asserting failure).

4.3 Case Study: HBB

On raw HBB pre-mRNA (1,608 bp, 3 exons, 2 introns), five predicates return FAIL: *SpliceCorrect* (cryptic donor at 620), *NoCrypticSplice* (donor at 620, acceptor at 943, both >3.0 bits), *NoRestrictionSite* (EcoRI at 287), *InFrame* (frame shift), *NoInstabilityMotif* (AUUUA at 1401). Two pass (GC=51.3%, CAI=0.73). These are real flaws consistent with known HBB errors [8].

4.4 Case Study: GFP

For eGFP (239 aa, 717 bp), six predicates yield PASS; *NoCrypticSplice* yields UNCERTAIN (14 donor sites scoring 3.19–4.69 bits—above the PASS threshold of 3.0 but below FAIL at 6.0). This is the correct three-valued outcome: weak cryptic sites cannot be eliminated by codon swapping alone, but are not strong enough for a definitive FAIL. CAI=0.9861, GC=61.7%. The certificate was independently re-verified.

The UNCERTAIN verdict arises when a measured value falls within a threshold margin; soundness is preserved since $\text{PASS} \sqcap \text{UNCERTAIN} = \text{UNCERTAIN} \neq \text{PASS}$.

5 Related Work

Computational gene design. NUPACK/Nuskell [4] optimizes nucleic-acid sequences without formal verification. DNA Chisel [10] checks constraints heuristically without certificates. Codon optimizers [11,12] lack compositional soundness.

Probabilistic splicing predictors. SpliceAI [13] predicts splice sites using deep learning but provides no formal guarantees—a single mis-prediction could yield a false PASS. BioCompiler asks whether a PASS verdict is *unconditionally* correct, trading coverage for certainty.

Formal verification in biology. Rashid and Hasan [5] verified nucleotide-counting in HOL4—analysis correctness, not type-system soundness. Cardelli [6] models biological systems without compiler soundness. Mjolsness [7] proposed “measurable types” without machine verification. BioCompiler is the first to apply Kleene logic to gene design with machine-verified soundness.

Verified compilers and OS kernels. CompCert [1] and seL4 [2] establish the methodology we follow: soundness conditional on explicit TCB assumptions. Our contribution is the *first application* of this methodology to gene design.

6 Conclusion

We presented BioCompiler, a compiler framework for gene design with a machine-verified soundness proof in Lean 4—zero **sorry**, zero axioms. The three-valued type system bridges biological non-determinism and formal methods: by reporting only what is *guaranteed*, we avoid modeling probability distributions while providing meaningful assurances. The central soundness theorem—every PASS verdict is correct—is compositional and independent of FFI output.

Future work. Validate TCB assumptions against wet-lab data; extend predicates to post-translational modifications; scale to multi-gene pathways (>10 kb).

References

1. X. Leroy, “A formally verified compiler back-end,” *J. Automated Reasoning*, vol. 43, no. 4, pp. 363–446, 2009.
2. G. Klein et al., “Comprehensive formal verification of an OS microkernel,” *ACM Trans. Computer Systems*, vol. 32, no. 1, pp. 1–70, 2014.
3. G. Yeo and C. Burge, “Maximum entropy modeling of short sequence motifs with applications to RNA splicing signals,” *J. Computational Biology*, vol. 11, no. 2–3, pp. 377–394, 2004.
4. J. Zadeh et al., “NUPACK: Analysis and design of RNA structures,” *J. Computational Chemistry*, vol. 32, pp. 170–173, 2011.

5. S. Rashid and K. Hasan, “Formal verification of DNA nucleotide counting algorithms using HOL4,” in *Proc. ICFEM*, Springer, 2017, pp. 293–308.
6. L. Cardelli, “On process rate semantics,” *Theoretical Computer Science*, vol. 391, no. 3, pp. 190–215, 2008.
7. E. Mjolsness, “Measurable types for gene expression,” *Bull. Mathematical Biology*, vol. 72, no. 2, pp. 313–333, 2010.
8. S. Thein, “Beta-thalassaemia,” *Baillière’s Clinical Haematology*, vol. 11, no. 1, pp. 91–126, 1998.
9. L. de Moura and N. Bjørner, “Z3: An efficient SMT solver,” in *Proc. TACAS*, Springer, 2008, pp. 337–340.
10. E. Valery et al., “DNA Chisel, a versatile genome sequence design optimizer,” *bioRxiv*, doi:10.1101/2020.03.17.995459, 2020.
11. A. Gouy and G. Gautier, “Codon optimization,” in *Encyclopedia of Bioinformatics and Computational Biology*, vol. 1, pp. 222–228, 2019.
12. F. Puigbò, E. Guzmán, A. Romeu, and S. Garcia-Vallvé, “OPTIMIZER: A web server for optimizing the codon usage of DNA sequences,” *Nucleic Acids Research*, vol. 35, pp. W126–W131, 2007.
13. K. Jaganathan et al., “Predicting splicing from primary sequence with deep learning,” *Cell*, vol. 176, no. 3, pp. 535–548, 2019.